

8. Linked List, Stack & Queue – Comprehensive Material

Module 1: Linked Lists – Operations at Speed

1.1 Singly Linked List – Core Operations

A singly linked list consists of nodes where each node has a `data` field and a `next` pointer to the next node.

Node definition:

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

Key operations with $O(1)$ or $O(n)$ time:

- **Insertion at head** – $O(1)$
- **Insertion at tail** – $O(n)$ without tail pointer; $O(1)$ with tail
- **Deletion by value** – $O(n)$ (search then remove)
- **Search** – $O(n)$

Code for basic operations (with tail pointer for $O(1)$ tail insert):

```
class SinglyLinkedList {
    ListNode head, tail;
```

```
int size;
```

```
// Insert at head
```

```
public void addFirst(int val) {  
    ListNode newNode = new ListNode(val);  
    if (head == null) {  
        head = tail = newNode;  
    } else {  
        newNode.next = head;  
        head = newNode;  
    }  
    size++;  
}
```

```
// Insert at tail
```

```
public void addLast(int val) {  
    ListNode newNode = new ListNode(val);  
    if (tail == null) {  
        head = tail = newNode;  
    } else {  
        tail.next = newNode;  
        tail = newNode;  
    }  
    size++;  
}
```

```
// Delete first occurrence of value
```

```
public boolean delete(int val) {  
    if (head == null) return false;  
    if (head.val == val) {  
        head = head.next;  
        if (head == null) tail = null;  
        size--;  
        return true;  
    }  
    ListNode curr = head;  
    while (curr.next != null && curr.next.val != val)  
        curr = curr.next;  
    }  
    if (curr.next == null) return false;
```

```

        curr.next = curr.next.next;
        if (curr.next == null) tail = curr;
        size--;
        return true;
    }
}

```

1.2 Doubly Linked List (DLL)

Each node has `prev` and `next` pointers, enabling **O(1) deletion of a known node** (given a reference) and easy backward traversal.

Node:

```

class DNode {
    int val;
    DNode prev, next;
    DNode(int val) { this.val = val; }
}

```

Operations (all O(1) if we have direct node reference):

- Insert at head, tail, before/after a given node.
- Delete a given node.

Implementation snippet (insert at head):

```

public void addFirst(int val) {
    DNode newNode = new DNode(val);
    if (head == null) {
        head = tail = newNode;
    } else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}

```

Delete a known node:

```
public void removeNode(DNode node) {
    if (node.prev != null) node.prev.next = node.next;
    else head = node.next;
    if (node.next != null) node.next.prev = node.prev;
    else tail = node.prev;
}
```

1.3 Circular Linked List

In a circular list, the **tail.next** points back to the **head**. This is useful for round-robin scheduling or buffering.

Code (singly circular):

```
class CircularList {
    ListNode head, tail;

    public void addLast(int val) {
        ListNode newNode = new ListNode(val);
        if (head == null) {
            head = tail = newNode;
            tail.next = head;    // point to itself
        } else {
            tail.next = newNode;
            tail = newNode;
            tail.next = head;    // maintain cycle
        }
    }

    public void print() {
        if (head == null) return;
        ListNode curr = head;
        do {
            System.out.print(curr.val + " ");
            curr = curr.next;
        } while (curr != head);
    }
}
```

```
        } while (curr != head);  
    }  
}
```

1.4 Pointer Manipulation Logic – Key Patterns

- **Reverse a linked list** (iterative and recursive).
 - **Find middle** (fast/slow).
 - **Detect cycle** (Floyd's algorithm).
 - **Merge two sorted lists**.
 - **Remove nth node from end** (two pointers).
 - **Split list** into two halves.
-

Module 2: Stack ADT & Applications

2.1 Stack Operations and Implementations

Stack is a **LIFO** structure. Operations:

- `push` (add top)
- `pop` (remove top)
- `peek` (view top)
- `isEmpty`

Implementation using `ArrayDeque` (preferred) or `LinkedList`:

```
Deque<Integer> stack = new ArrayDeque<>();  
stack.push(10);  
int top = stack.pop();
```

2.2 Expression Evaluation using Stack

a) Infix to Postfix (Shunting-Yard algorithm)

Idea: Use stack for operators. Output numbers directly; operators with higher precedence pop lower ones.

Code:

```
public String infixToPostfix(String expr) {
    StringBuilder output = new StringBuilder();
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Integer> precedence = Map.of('+', 1, '-'

    for (char c : expr.toCharArray()) {
        if (Character.isLetterOrDigit(c)) {
            output.append(c);
        } else if (c == '(') {
            stack.push(c);
        } else if (c == ')') {
            while (!stack.isEmpty() && stack.peek() != '(')
                output.append(stack.pop());
            stack.pop(); // remove '('
        } else { // operator
            while (!stack.isEmpty() && precedence.getOrDefault(stack.peek(), 0) > precedence.getOrDefault(c, 0))
                output.append(stack.pop());
            stack.push(c);
        }
    }
    while (!stack.isEmpty()) output.append(stack.pop());
    return output.toString();
}
```

b) Evaluate Postfix Expression

Idea: Use stack to store operands. When an operator appears, pop two operands, compute, push result.

```

public int evaluatePostfix(String expr) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (char c : expr.toCharArray()) {
        if (Character.isDigit(c)) {
            stack.push(c - '0');
        } else {
            int b = stack.pop();
            int a = stack.pop();
            switch (c) {
                case '+': stack.push(a + b); break;
                case '-': stack.push(a - b); break;
                case '*': stack.push(a * b); break;
                case '/': stack.push(a / b); break;
            }
        }
    }
    return stack.pop();
}

```

c) Balanced Parentheses

```

public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') stack.push(c);
        else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == ']' && top != '[') return false;
            if (c == '}' && top != '{') return false;
        }
    }
    return stack.isEmpty();
}

```

Module 3: Queue ADT & Applications

3.1 Queue Operations

Queue is **FIFO**. Operations:

- `offer` / `add` (enqueue)
- `poll` / `remove` (dequeue)
- `peek` (view front)

Implementations: `LinkedList`, `ArrayDeque`, `PriorityQueue`.

3.2 Circular Queue

A fixed-size queue that reuses space by wrapping around using modulo arithmetic.

```
class CircularQueue {
    private int[] arr;
    private int front, rear, size, capacity;

    public CircularQueue(int cap) {
        capacity = cap;
        arr = new int[cap];
        front = 0;
        rear = -1;
        size = 0;
    }

    public boolean enqueue(int val) {
        if (isFull()) return false;
        rear = (rear + 1) % capacity;
        arr[rear] = val;
        size++;
        return true;
    }
}
```



```

public int dequeue() {
    if (isEmpty()) return -1;
    int val = arr[front];
    front = (front + 1) % capacity;
    size--;
    return val;
}

public boolean isFull() { return size == capacity; }
public boolean isEmpty() { return size == 0; }
}

```

3.3 Deque (Double-Ended Queue)

Allows insertion/removal from both ends. `ArrayDeque` is the recommended implementation.

Operations: `addFirst`, `addLast`, `removeFirst`, `removeLast`, `peekFirst`, `peekLast`.

Module 4: Sliding Window Maximum using Deque

Problem: Given an array and window size `k`, find the maximum in each window.

Idea: Use a `Deque` to store **indices** of elements in the current window. Maintain the deque such that it is **decreasing** (head is max). When sliding, remove indices outside the window, and remove smaller elements from the back (because they will never be max).

Algorithm:

1. For first window, add indices while maintaining decreasing order.
2. For each subsequent window, remove if front index is out of

window, then add new element while popping smaller from back.
The front is the max.

Implementation:

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums.length == 0 || k == 0) return new int[0];
    int[] result = new int[nums.length - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();

    for (int i = 0; i < nums.length; i++) {
        // remove indices outside window
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1)
            deque.pollFirst();

        // remove smaller elements from back (they are useless)
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i])
            deque.pollLast();

        deque.offerLast(i);

        // start recording after first window is filled
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }

    return result;
}
```

Complexity: $O(n)$ time, $O(k)$ space.

Module 5: LRU and LFU Cache Design

5.1 LRU Cache (Least Recently Used)

Data structure: `HashMap` (key $\rightarrow$ node) + `Doubly Linked List` (to

maintain access order).

- On `get(key)`: move node to head (most recent).
- On `put(key, value)`: if exists, update value and move to head; else add new node; if capacity exceeded, remove from tail (least recent).

Node (DLL):

```
class Node {
    int key, val;
    Node prev, next;
    Node(int key, int val) { this.key = key; this.val = v
}
}
```

LRU implementation:

```
class LRUCache {
    private final int capacity;
    private final Map<Integer, Node> map;
    private final Node head, tail;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        map = new HashMap<>();
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node);
        insertAfterHead(node);
        return node.val;
    }
}
```

```

public void put(int key, int value) {
    if (map.containsKey(key)) {
        Node node = map.get(key);
        node.val = value;
        remove(node);
        insertAfterHead(node);
    } else {
        if (map.size() == capacity) {
            Node toRemove = tail.prev;
            remove(toRemove);
            map.remove(toRemove.key);
        }
        Node newNode = new Node(key, value);
        map.put(key, newNode);
        insertAfterHead(newNode);
    }
}

private void remove(Node node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private void insertAfterHead(Node node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}
}

```

Time: $O(1)$ for get/put.

5.2 LFU Cache (Least Frequently Used)

More complex: maintain frequency buckets (often using a `Map<Integer, LinkedHashSet<Node>>` for each frequency) and a min-heap or a frequency map. A common approach uses two maps: one

for key→node, one for frequency→DLL (or LinkedHashSet) to track order.

Simpler approach using `Map<Integer, Node>` with frequency and `LinkedHashSet` to maintain insertion order per frequency:

```
class LFUCache {
    private final int capacity;
    private int minFreq;
    private final Map<Integer, Node> keyToNode;
    private final Map<Integer, LinkedHashSet<Node>> freqToNodes;

    static class Node {
        int key, val, freq;
        Node(int key, int val) { this.key = key; this.val = val; }
    }

    public LFUCache(int capacity) {
        this.capacity = capacity;
        this.minFreq = 0;
        keyToNode = new HashMap<>();
        freqToNodes = new HashMap<>();
    }

    public int get(int key) {
        if (!keyToNode.containsKey(key)) return -1;
        Node node = keyToNode.get(key);
        updateNode(node);
        return node.val;
    }

    public void put(int key, int value) {
        if (capacity == 0) return;
        if (keyToNode.containsKey(key)) {
            Node node = keyToNode.get(key);
            node.val = value;
            updateNode(node);
        } else {
            if (keyToNode.size() == capacity) {
                // Remove the least frequent node
                int minKey = null;
                int minFreq = minFreq;
                for (Node node : keyToNode.values()) {
                    if (node.freq == minFreq) {
                        minKey = node.key;
                    }
                }
                keyToNode.remove(minKey);
            }
            Node node = new Node(key, value);
            keyToNode.put(key, node);
            freqToNodes.computeIfAbsent(node.freq, k -> new LinkedHashSet<>()).add(node);
            minFreq = node.freq;
        }
    }

    private void updateNode(Node node) {
        LinkedHashSet<Node> freqSet = freqToNodes.get(node.freq);
        freqSet.remove(node);
        if (freqSet.isEmpty()) {
            freqToNodes.remove(node.freq);
        }
        node.freq++;
        freqToNodes.computeIfAbsent(node.freq, k -> new LinkedHashSet<>()).add(node);
        minFreq = Math.max(minFreq, node.freq);
    }
}
```

```

        // evict least frequent (minFreq) and leave
        LinkedHashSet<Node> set = freqToList.get(minFreq);
        Node toEvict = set.iterator().next();
        set.remove(toEvict);
        keyToNode.remove(toEvict.key);
        if (set.isEmpty()) freqToList.remove(minFreq);
    }
    Node newNode = new Node(key, value);
    keyToNode.put(key, newNode);
    freqToList.computeIfAbsent(1, k -> new LinkedHashSet<Node>());
    minFreq = 1;
}

private void updateNode(Node node) {
    int freq = node.freq;
    LinkedHashSet<Node> set = freqToList.get(freq);
    set.remove(node);
    if (set.isEmpty()) {
        freqToList.remove(freq);
        if (minFreq == freq) minFreq++;
    }
    node.freq++;
    freqToList.computeIfAbsent(node.freq, k -> new LinkedHashSet<Node>());
}
}

```

Time: $O(1)$ amortized for get/put.

Module 6: Hands-on Problems & Solutions

6.1 Reversal of Linked List

Problem: Reverse a singly linked list iteratively.

Solution: Standard three-pointer method.

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Recursive:

```
public ListNode reverseRecursive(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

6.2 Cycle Detection

Problem: Detect if a linked list has a cycle.

Solution: Floyd's Cycle Detection (fast/slow).

```
public boolean hasCycle(ListNode head) {
    if (head == null || head.next == null) return false;
    ListNode slow = head, fast = head.next;
    while (slow != fast) {
        if (fast == null || fast.next == null) return false;
        slow = slow.next;
        fast = fast.next.next;
    }
    return true;
}
```

```
    }  
    return true;  
}
```

6.3 Stack-based Expression Problems

- **Evaluate Reverse Polish Notation (postfix)** – already covered.
- **Infix to Postfix** – covered.
- **Basic Calculator** (with parentheses, +, -, *, /).

Example: Evaluate expression like "3+2*2" using two stacks (one for numbers, one for operators) or convert to postfix.

6.4 Queue-based Sliding Window Problems

- **Maximum in sliding window** – covered using deque.
- **First negative integer in every window of size k.**
- **Longest substring without repeating characters** – sliding window with HashSet.

Practice Exercises (10 problems)

1. **Reverse Nodes in k-Group** (LeetCode 25) – Hard but builds pointer skills.
2. **Merge Two Sorted Lists** (LeetCode 21) – Easy.
3. **Detect Cycle and return the node** (LeetCode 142) – Medium.
4. **Design a Min Stack** (LeetCode 155) – Stack with $O(1)$ getMin.
5. **Implement Queue using Stacks** (LeetCode 232) – Easy.
6. **Evaluate Reverse Polish Notation** (LeetCode 150) – Medium.
7. **Sliding Window Maximum** (LeetCode 239) – Hard (already covered).
8. **LRU Cache** (LeetCode 146) – Medium (code above).
9. **LFU Cache** (LeetCode 460) – Hard (code above).
10. **Circular Queue design** – implement class with enqueue/

dequeue.

Summary Table

Topic	Key Data Structure	Common Operations	Complexity
Singly Linked List	<code>ListNode</code>	insert/delete/search	$O(1)$ at head, $O(n)$ elsewhere
Doubly Linked List	<code>DNode</code>	insert/delete (given node)	$O(1)$ if node known
Circular List	<code>ListNode</code> with <code>tail</code> $\rightarrow$ <code>head</code>	round-robin, buffer	$O(1)$ insert at tail
Stack	<code>Deque</code>	push/pop/peek	$O(1)$
Queue	<code>Deque</code> or <code>LinkedList</code>	offer/poll/peek	$O(1)$
Deque	<code>ArrayDeque</code>	addFirst/Last, removeFirst/Last	$O(1)$
Sliding Window Max	<code>Deque</code> (indices)	maintain decreasing order	$O(n)$ total
LRU Cache	<code>HashMap</code> + <code>DLL</code>	get/put	$O(1)$
LFU Cache	<code>HashMap</code> + <code>HashMap</code> <freq, <code>LinkedHashSet</code> >	get/put	$O(1)$ amortized

This material covers all the requested topics with detailed implementations, complexity, and hands-on problems. Practice these

thoroughly to master linked lists, stacks, and queues for competitive programming and interviews